Sous: the *zero-waste* kitchen agent for Alexa+

Sous keeps everyone at the table safe and fed: a child with a life-threatening food allergy, a grandparent on a low-sodium diet, a family on a tight budget, and the food that would otherwise be thrown away. A self-hosted MCP server (spec 2025-11-25, Streamable HTTP) with 44 tools, six MCP Apps cards for Echo Show (including a full-screen allergy emergency alert) and an Amazon Bedrock agent.



LIVE DEMO
sous-alexa.vercel.app/device

CODE (MIT)
github.com/johnbosco-j/Sous

MCP ENDPOINT
https://sous-alexa.vercel.app/mcp

TRACKS
Alexa+ (primary) · AWS Builder · Open Source

DEPTH
44 MCP tools · 254 automated tests

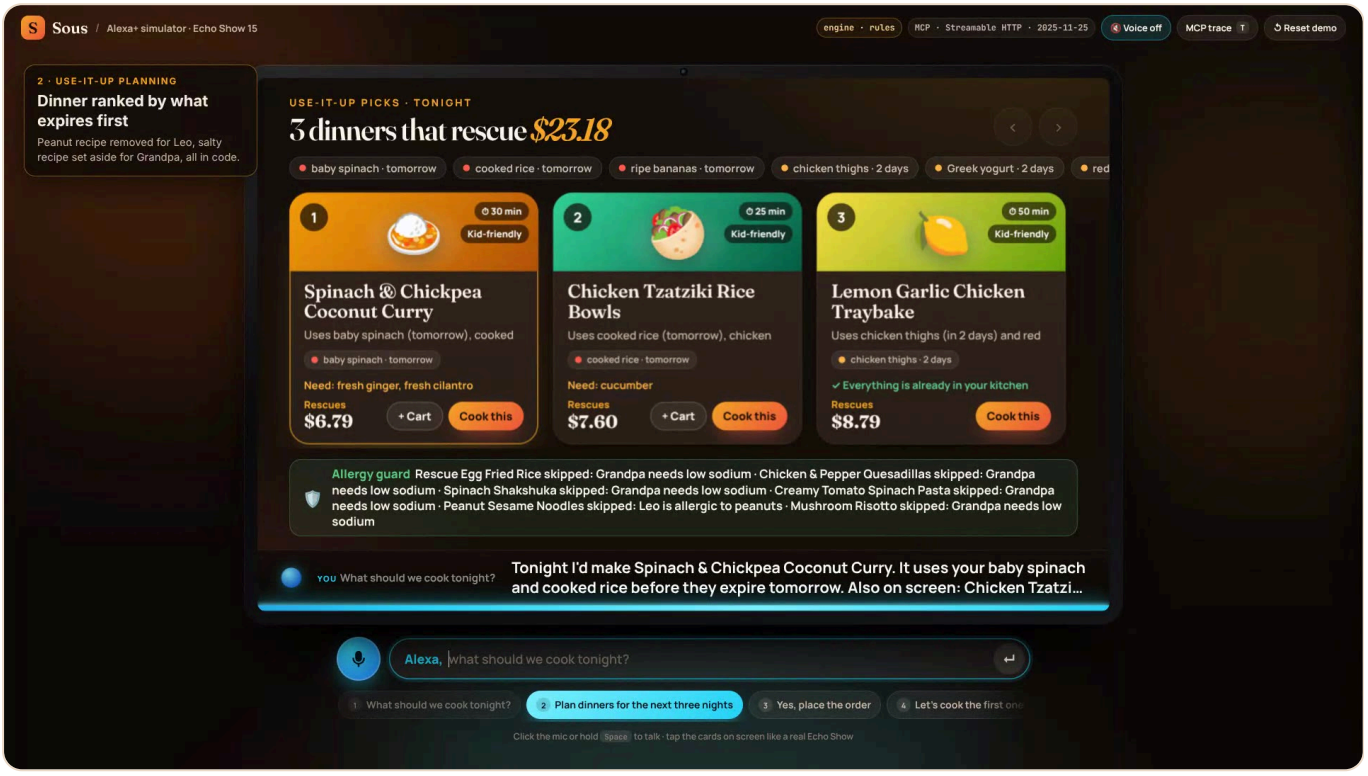
DEMO VIDEO
2:53 walkthrough on YouTube (see Devpost)

BUILT
23 October 2026, during the submission window

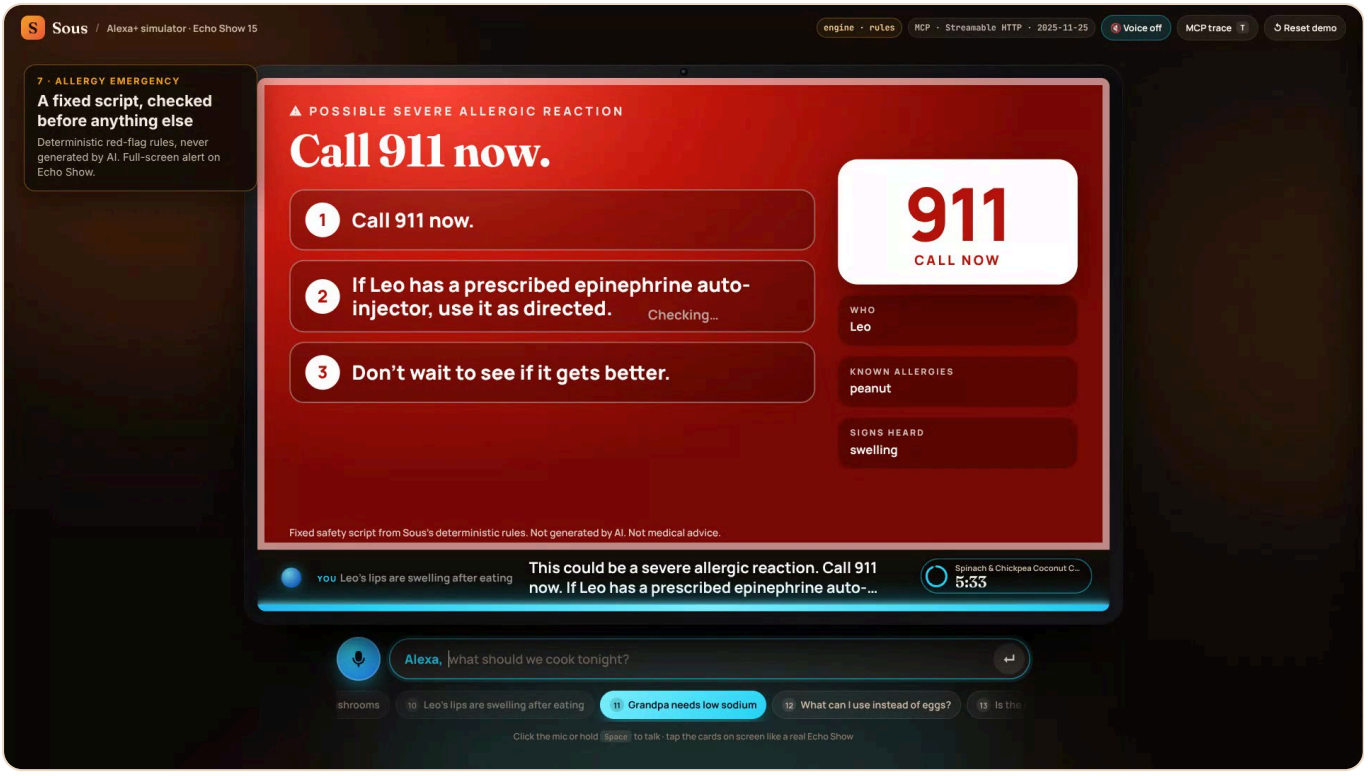
Screenshots



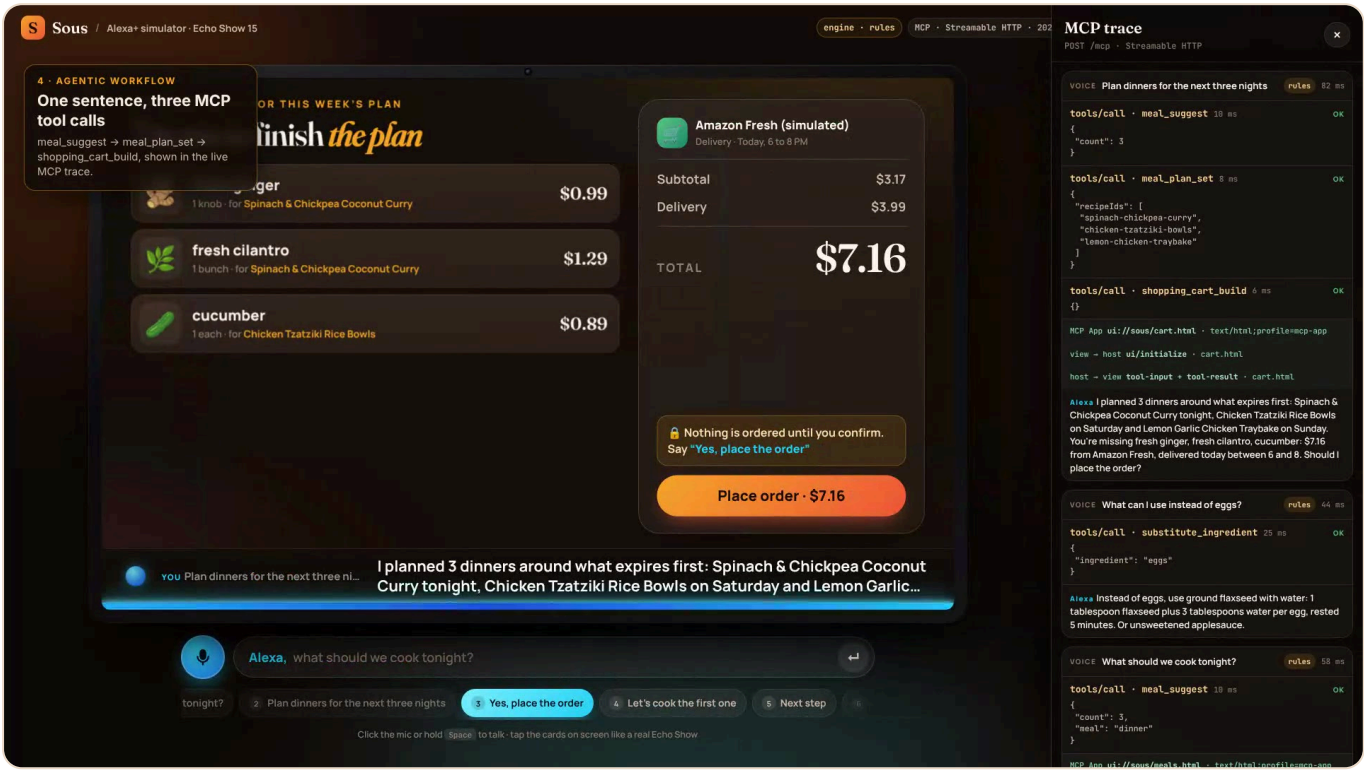
1. Landing page: "Sous: the zero-waste kitchen agent for Alexa+." Keeps everyone at the table safe and fed.



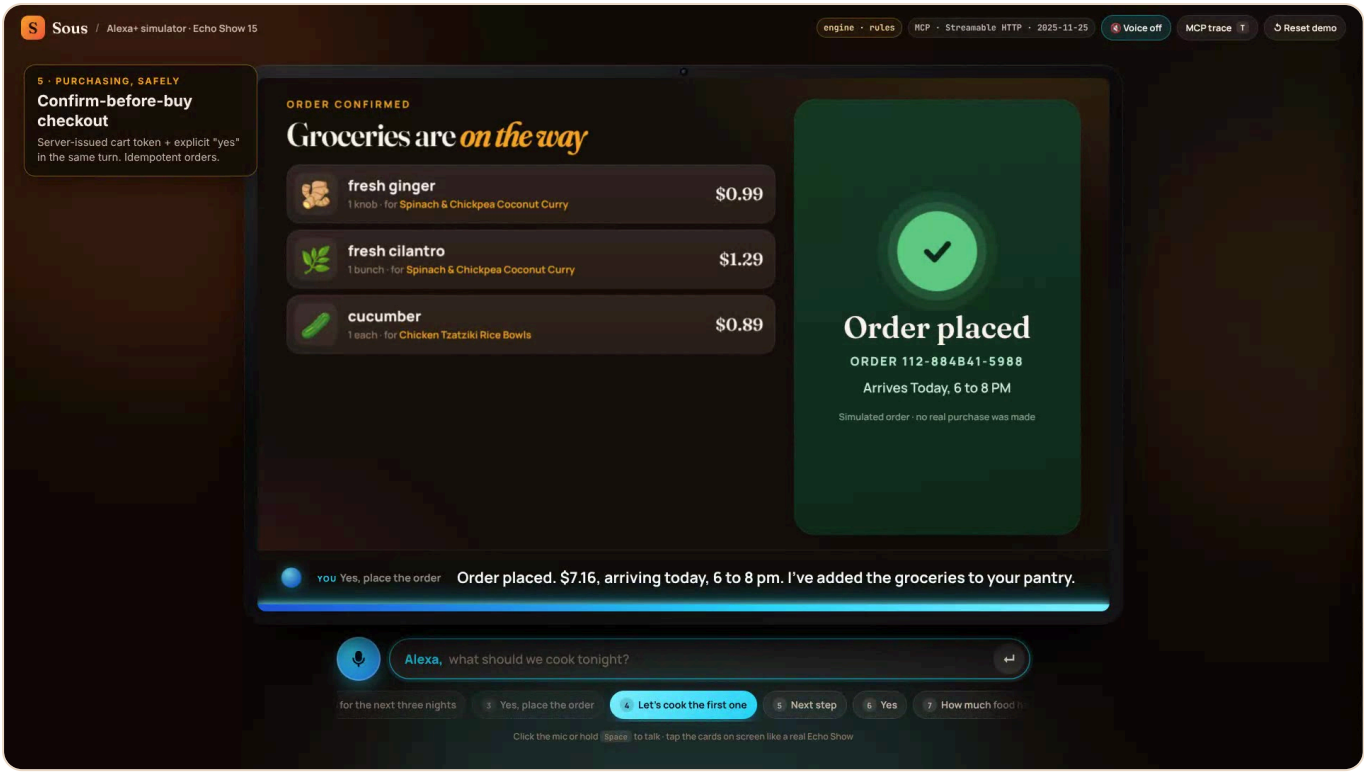
2. Use-it-up planning, safe for everyone: recipes ranked by the expiring food they rescue. The allergy and diet guard sets recipes aside with reasons ("Leo is allergic to peanuts", "Grandpa needs low sodium").



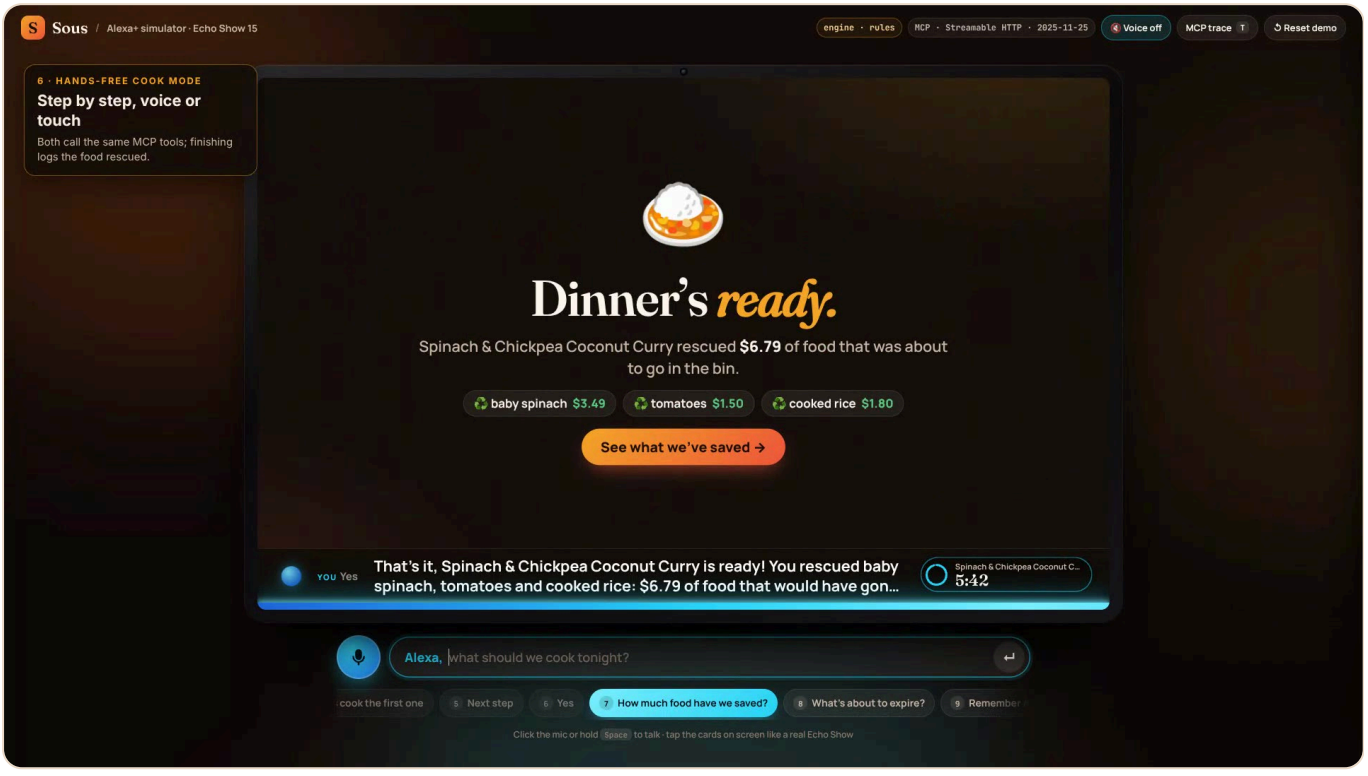
3. Allergy emergency: "Leo's lips are swelling" triggers a fixed, deterministic script on a full-screen Echo Show alert: call 911, use the prescribed epinephrine, don't wait. Checked before anything else, never generated by AI.



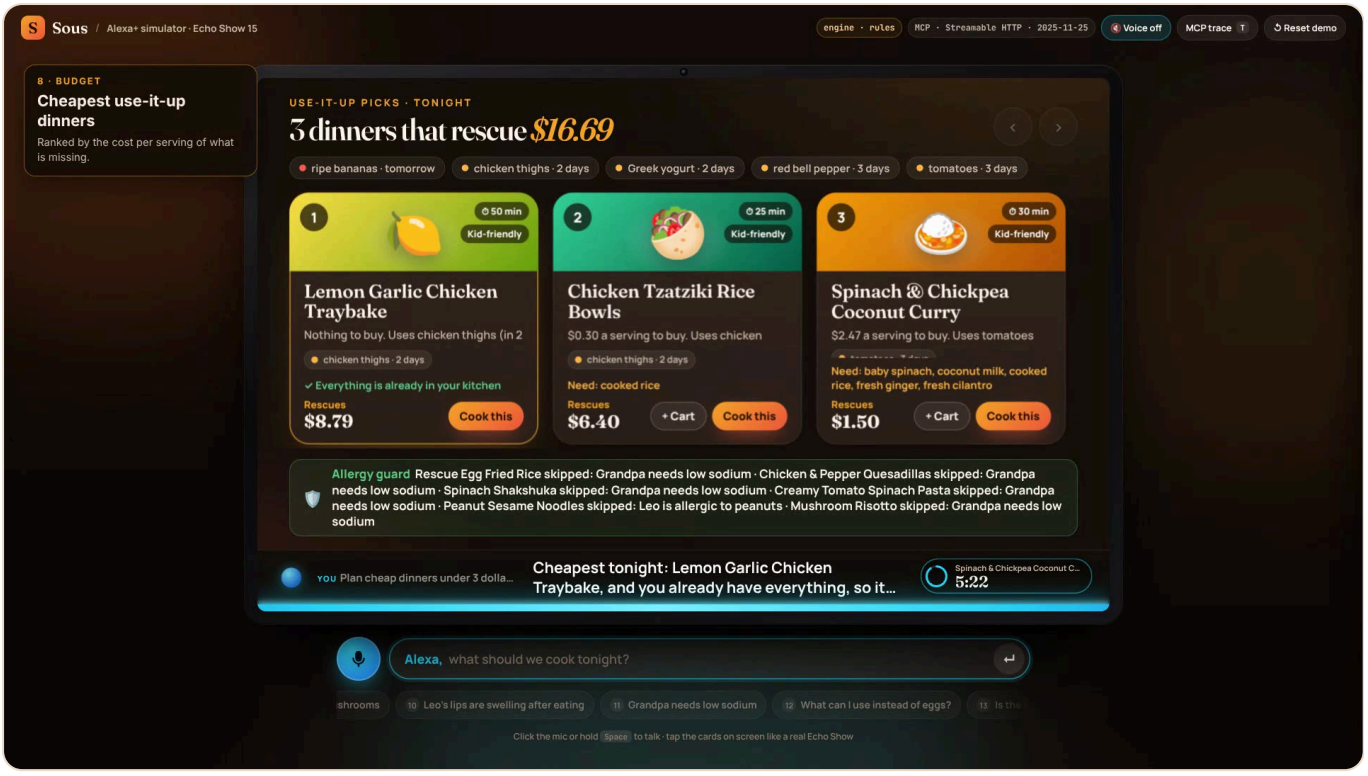
4. Agentic workflow: one sentence becomes three MCP tool calls (meal_suggest → meal_plan_set → shopping_cart_build), visible in the live MCP trace.



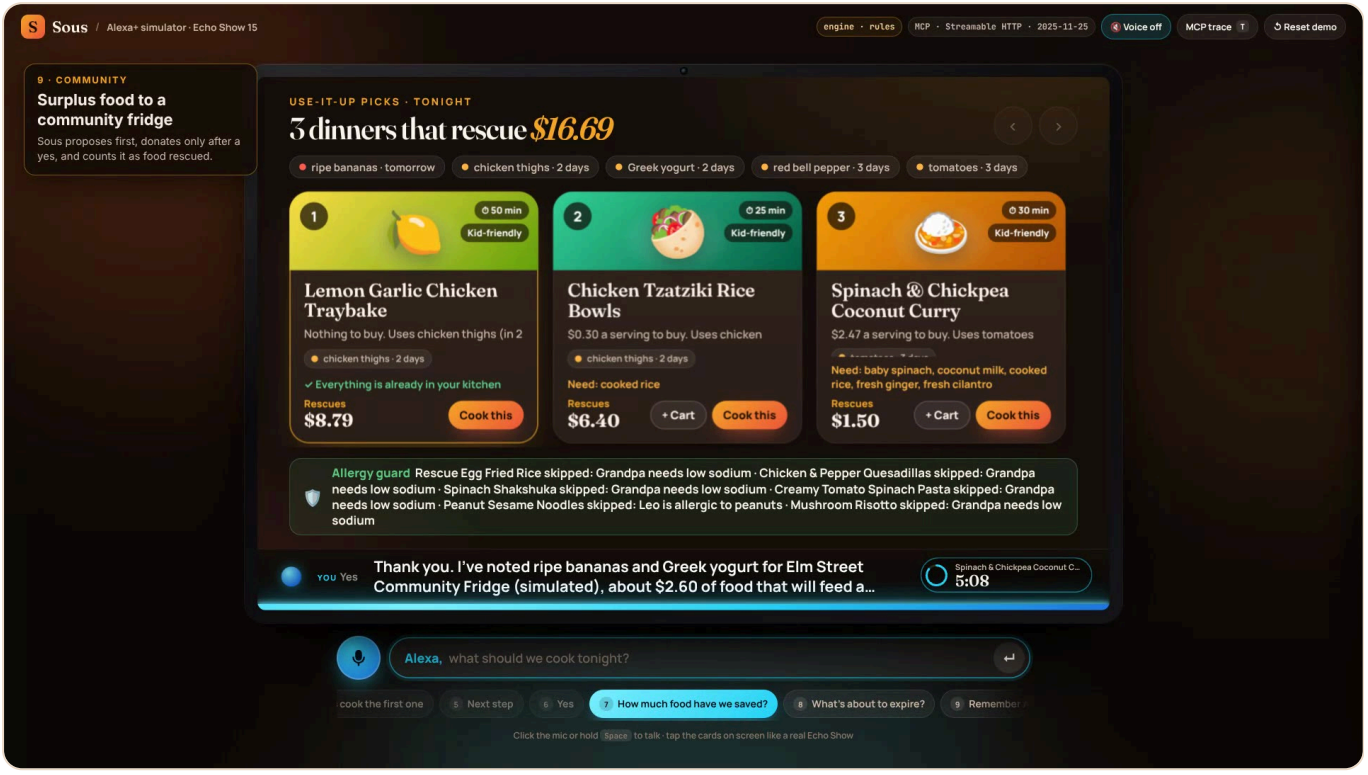
5. Confirm-before-buy: the order is placed only after an explicit "yes" with the server-issued cart token. Idempotent; simulated Amazon Fresh.



6. Hands-free cook mode by voice or touch. Finishing records exactly which expiring ingredients were rescued.



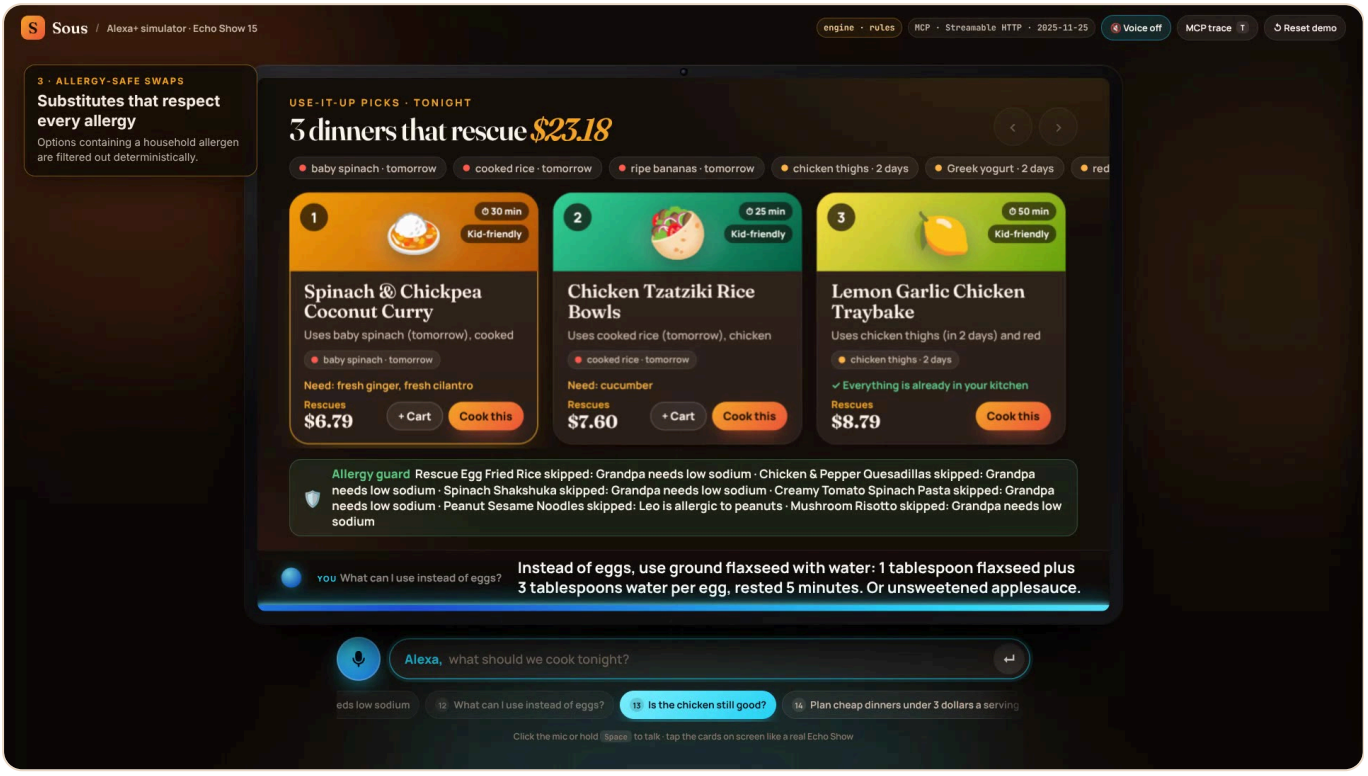
7. Budget: the cheapest use-it-up dinners, ranked by the cost per serving of what is missing.



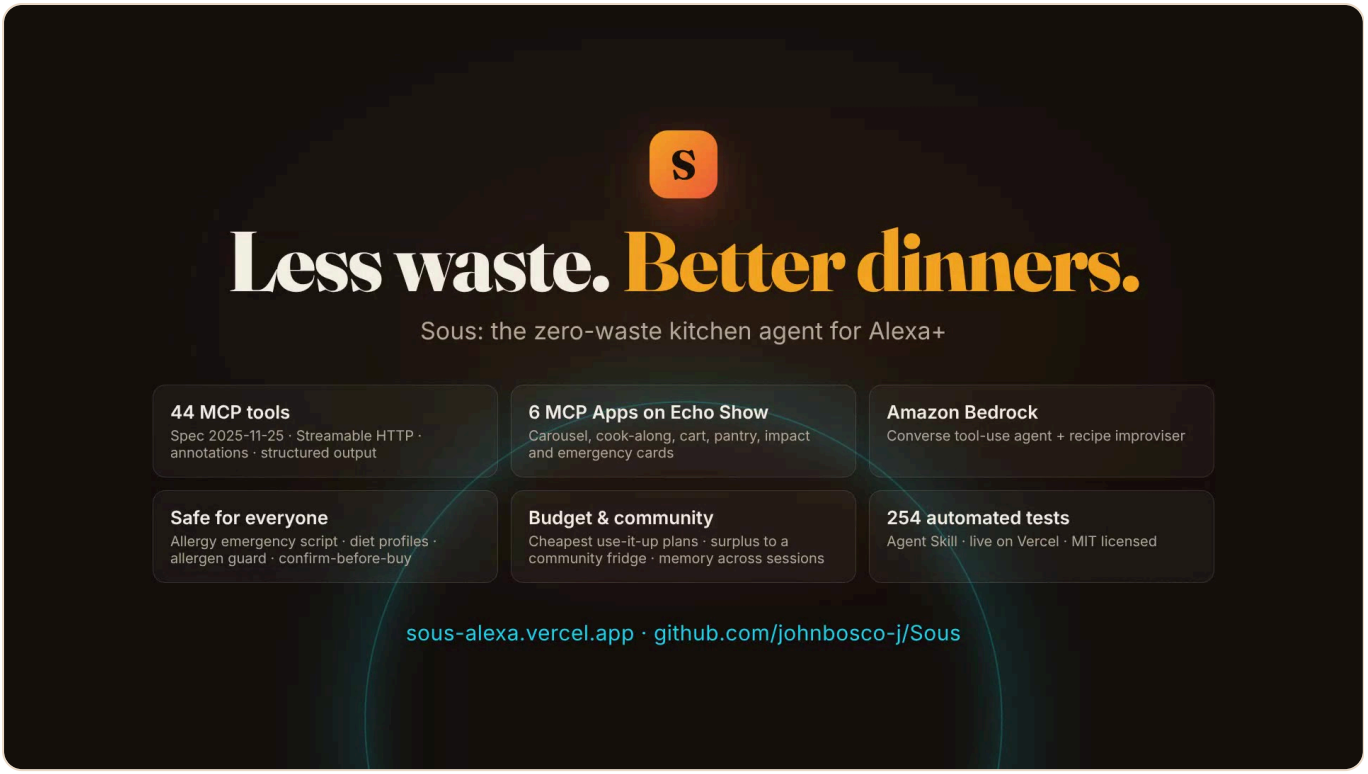
8. Community: surplus food goes to a community fridge (simulated) only after a yes, and counts as food rescued.



9. Impact: dollars, kilograms and CO2e rescued this week (cooking and donations), with a yearly projection.



10. Allergy-safe substitutes: options that clash with any household allergy are filtered out deterministically.



11. Summary: 44 MCP tools, 6 MCP Apps views, Amazon Bedrock, safe for everyone, budget and community, 254 automated tests.

Project name

Sous

Tagline / elevator pitch (Devpost limit: 200 chars)

Alexa+ kitchen agent that keeps everyone at the table safe and fed: dinner from food about to expire, allergy and diet risks blocked in code, groceries ordered only after a yes.

Short summary (use where a longer summary is allowed)

Sous is a self-hosted MCP server (spec 2025-11-25, Streamable HTTP) that turns Alexa+ on an Echo Show into a kitchen agent for the whole family, built on one promise: **everyone at the table is safe and fed, and good food doesn't go in the bin.** It plans dinner around food that's about to expire. It removes recipes that clash with a child's food allergy or a grandparent's low-sodium diet, in deterministic code. If someone describes a severe allergic reaction, it gives a fixed emergency script. It keeps the weekly grocery budget and orders only after an explicit "yes". It cooks with you hands-free on interactive MCP Apps cards, and it shows the dollars and CO2e you rescued. 44 MCP tools, 254 automated tests, live at https://sous-alexa.vercel.app.

About the project (full description)

Inspiration

Dinner is a safety decision, a money decision and a waste decision, made every day around 6 p.m. with your hands full.

- **Food allergies.** More than **33 million people in the US live with a food allergy, about 1 in 13 children** (FARE National Indicator Report), and FARE reports that a food allergy reaction sends someone to the emergency room **every 10 seconds** (FARE). For those families, one wrong ingredient can mean an emergency room visit.
- **Older family members' diets. 47.7% of US adults have high blood pressure, rising to 71.6% of adults 60 and older** (CDC NCHS Data Brief 511). In multigenerational homes, the same pot has to work for a grandparent on a low-sodium diet and a kid who only eats pasta.
- **Tight budgets, wasted food. 13.5% of US households, about 18 million, were food insecure at some point in 2023** (USDA ERS, ERR-337). Meanwhile the USDA estimates that **30–40% of the US food supply is wasted**. ReFED's 2025 report puts consumer food waste at **~35 million tons and $261 billion a year, nearly $800 per person**, and surplus food drives roughly **4% of US greenhouse-gas emissions**.

These are often the same families at the same moment. Nobody can hold in their head what's expiring, who can't eat what, and what's left in the budget while a toddler pulls at their sleeve. That is a coordination failure, not a moral one. Coordination is exactly what an agent with memory and tools is good at.

The kitchen is where voice belongs. Amazon's own Alexa+ add-on design guide names kitchens as a core context, with customers "often multitasking, in kitchens, living rooms, and cars." So we built Sous: the agent between your fridge, your family's needs and Alexa+.

What it does

Sous is an **Alexa+ add-on implemented as a self-hosted MCP server**. Alexa+ (or any MCP client) calls its tools. On screen devices, Sous renders **MCP Apps** views: interactive cards and carousels on the Echo Show.

A typical evening with the demo household, the Riveras (Leo, 8, is allergic to peanuts; Ava, 5, won't eat mushrooms):

1. **"Alexa, what should we cook tonight?"** `meal_suggest` ranks recipes by a *use-it-up score*:

   - each ingredient is weighted by how close it is to expiry (today 6, tomorrow 5, two days 3, three days 2, four days 1), scaled by how much of that at-risk stock the recipe clears;
   - every missing ingredient costs points, and long recipes cost a little too;
   - a recipe that contains a member's allergen or dislike is **set aside, with the reason shown**.

   The Echo Show shows a carousel of cards like *"Spinach & Chickpea Coconut Curry: uses baby spinach and cooked rice (expire tomorrow)"*, each with the dollar value it rescues. Peanut Sesame Noodles is **removed**, and the card says why: "Leo is allergic to peanuts."

2. **"Remember Ava doesn't like mushrooms."** `household_memory` stores it, and it persists across sessions and restarts.

3. **"Add what we're missing to the cart."** (or **"Plan dinners for the next three nights"**, which chains `meal_suggest` → `meal_plan_set` → `shopping_cart_build`). `shopping_cart_build` compares the recipes with the pantry and rounds each item up to a whole pack. It returns an itemized cart from a simulated Amazon Fresh catalog and a `cartToken`.

4. **"Yes, place the order."** `shopping_checkout` will **only** place an order with that token *and* `confirmed=true`. Without confirmation, it returns a `followUp` asking for it. Carts expire after 30 minutes, so prices can't go stale. Replaying a placed cart returns the same order and never a second one. The purchased groceries go straight into the pantry. The model cannot buy anything on its own.

5. **"Let's cook the first one." / "Next step." / "Set a timer for 10 minutes."** `cook_start`, `cook_step` and `timer_set` run a hands-free cook mode. Each step gets a giant-type card, with step dots, an ingredient checklist and live timers.

6. When you finish, the ingredients you used are deducted from the pantry. `impact_report` then shows **dollars rescued, kilograms of food saved, kg CO2e avoided** and a yearly projection.

7. **"What can I make with yogurt, peppers and tortillas?"** `chef_improvise` asks **Amazon Bedrock** for a new recipe. A *deterministic* allergen guard checks the generated recipe before it is shown. If it contains an allergen, it is rejected.

Safety for every person at the table

- **"Leo's lips are swelling after dinner."** `allergy_emergency` recognizes signs of a severe allergic reaction: swelling of the lips, tongue or throat, trouble breathing, wheezing, or hives after eating. It handles negations such as "no trouble breathing". It answers with a **fixed script written in code, never by a model**: call 911 now, and use the prescribed epinephrine auto-injector as directed. Milder signs get a fixed "watch closely; if there's any swelling or trouble breathing, call 911" line. Sous does not diagnose or reassure, and every incident is logged for the family.
- **"Does this granola have anything Leo can't eat?"** `allergen_scan` reads a product's ingredient label, including "may contain" and shared-facility warnings, and names the household member it affects.
- **"Grandpa needs low-sodium meals."** `diet_profile` stores per-member diet needs (low-sodium, diabetes-friendly, vegetarian, soft-foods, dairy-free, gluten-free) across sessions. From then on `meal_suggest` sets aside recipes that clash, with the reason shown.
- **Other safety tools:**
  - `meal_safety_check` gives a per-person verdict for any recipe, covering allergens, diet and dislikes.
  - `cross_contact_tips` explains how to keep a meal allergy-safe: separate boards and utensils, cook the allergy-safe dish first, and re-read labels.
  - `cooking_temp` gives USDA safe minimum internal temperatures, for example poultry at 165°F (74°C).
  - `food_safety_check` gives conservative storage windows and never declares food "safe".
  - `substitute_ingredient` only offers swaps that avoid every household allergen.

Budget and community

- `budget_set` and `budget_status` track a weekly grocery budget against placed orders. `budget_meal_plan` picks use-it-up dinners that fit it, and `price_check` answers "how much is feta?".
- `leftovers_log` tracks cooked leftovers against the USDA 3–4 day window, and `leftovers_ideas` turns them into tomorrow's lunch.
- `donate_surplus` finds food you won't get to before it expires and prepares a hand-off to a community fridge (simulated listings in this build), so it feeds a neighbor instead of the bin.
- `waste_log` and `waste_insights` keep the impact numbers honest: what still went in the bin, and which categories you waste most.

Planning and household helpers

- **Recipes and plans:** `recipe_search`, `recipe_details` (scaled servings plus an allergen check), `meal_plan_get` and `meal_plan_clear`.
- **Cooking:** `unit_convert`, `timer_list`, `timer_cancel` and `cook_status`.
- **Shopping:** `shopping_list` (it feeds the cart), `order_history`, `reorder_staples` (through the same purchase gate) and `delivery_slots`.
- **Household:** `household_members`, `pantry_add_receipt` (paste a receipt and the pantry updates) and `daily_brief` (what to use today).

How it works (architecture)

```
 Voice / touch
     |
 Alexa+ (or the Sous Echo Show simulator)  ← agent: Amazon Bedrock Converse API with tool
use
     |  MCP 2025-11-25 over Streamable HTTP (POST/GET/DELETE /mcp, Mcp-Session-Id)
     ▼
 Sous MCP server (Node.js + TypeScript, @modelcontextprotocol/sdk)
   ├─ 44 tools in modules: core kitchen · safety (allergy emergency, labels, diets, temps) ·
budget & community · planning · household
   ├─ ui:// resources (MCP Apps, text/html;profile=mcp-app) → Echo Show cards & carousels
   ├─ deterministic guards: allergen filter, purchase confirmation token, quantity/unit
sanity
   └─ household state persisted to disk (state across sessions)
```

MCP implementation details

- **Protocol 2025-11-25** over **Streamable HTTP**, with session management through `Mcp-Session-Id`.
- All 44 tools declare a `title`, a Zod-generated `inputSchema`, an `outputSchema`, **icons** (new in 2025-11-25, SEP-973), and behavior **annotations**:
  - `pantry_check`, `meal_suggest` and `impact_report` are `readOnlyHint`;
  - `shopping_checkout` is `openWorldHint` + `idempotentHint`, because replaying a token never orders twice;
  - `chef_improvise` is `openWorldHint`, because it calls Bedrock.
- Resources `sous://household`, `sous://pantry` and `sous://impact`, a resource template `sous://recipe/{id}`, and prompts `plan_tonight` and `use_it_up_week`.
- Every result carries a short voice-safe `text` block **and** `structuredContent`. The structured envelope is `{ ok, tool, speech, view, data, followUp }`. `followUp` hands the agent the *exact* next call for confirmations, so "yes" on the next turn never depends on the model reconstructing arguments from speech.
- **MCP Apps.** UI-bearing tools link a `ui://sous/<view>.html` resource through `_meta.ui.resourceUri`. There are six views: pantry, meals carousel, cook mode, cart/checkout, impact, and a full-screen allergy emergency alert. They speak the MCP Apps postMessage protocol: `ui/initialize` → `ui/notifications/initialized` → `ui/notifications/tool-input` / `tool-result`. Buttons inside a card ("Cook this", "Add to cart", "Next") send `tools/call` back through the host, so touch and voice drive the same tools.
- **Server** `instructions` tell the agent how to behave: speak the tool's text, ask the follow-up prompt, and never place an order without an explicit yes.
- An **Agent Skill** (`skills/sous-kitchen/SKILL.md`) packages the same conversation contract for any agent that supports Agent Skills.

The Alexa+ simulation. Real Alexa+ add-on testing (the MCP Toolkit's web simulator and inspector) is in Private Preview. We therefore built an **Echo Show-style simulator** (`/device`) that acts as an MCP Apps host and runs the agent loop:

- Voice in (Web Speech API), voice out (speech synthesis), the Alexa light bar, and a sandboxed iframe for MCP Apps views.
- A live **MCP trace drawer** shows every `tools/call` with its arguments and latency, so judges can see the protocol working.

- The agent's brain is **Amazon Bedrock**, using the Converse API with tool use. Sous's MCP tool list is mapped to Bedrock `toolSpec` s. The loop runs `converse` → `toolUse` → MCP `tools/call` → `toolResult` → final spoken answer.
- With no AWS credentials, a deterministic intent router takes over, so the demo runs anywhere.

Safety and trust design

These rules live in deterministic, tested code, not in a prompt. A model can phrase things, but it cannot override them.

- **Allergy-emergency red flag.** `allergy_emergency` matches the signs of a severe reaction, with negation handling. It returns a fixed escalation script: call 911, and use the prescribed epinephrine auto-injector as directed. The text is a constant in code, so no model writes it, and the server `instructions` tell Alexa to speak it as written.
- **Allergen guard everywhere.** Recipes containing a household member's allergen are excluded before ranking, and the reason is shown ("Leo is allergic to peanuts"). The same guard runs on:
  - LLM-generated recipes from `chef_improvise`;
  - substitutes;
  - scanned product labels (`allergen_scan`), including "may contain" warnings.
- **Diet needs as hard filters.** `diet_profile` plugs into the planner through a recipe-filter hook, so a grandparent's low-sodium need removes recipes exactly as an allergy does, and says why.
- **Food safety without false reassurance.**
  - `food_safety_check` never says food is safe. It gives the date on record, the general fridge window and "when in doubt, throw it out", and raw meat past its date is always a discard.
  - `cooking_temp` returns USDA safe minimum internal temperatures.
  - `cross_contact_tips` covers the prep steps that keep an allergy-safe meal safe.
- **Confirm-before-buy, enforced twice.** Every purchase path (`shopping_checkout`, and `reorder_staples` through it) goes through the same gate.
  - The server requires the `cartToken` from the cart the user just saw plus `confirmed=true`, and it rejects stale (over 30 minutes) or unknown carts.
  - The agent host adds a second gate. If the model calls `shopping_checkout` with `confirmed=true` but the user's latest turn contained no explicit consent, the host downgrades the call to `confirmed=false`. Sous then asks instead of buying. A test proves this.
  - Orders are clearly labeled *simulated*.
- **Grounded answers.** Expiry dates, prices and quantities come from the pantry and catalog, never from the model. Impact uses a fixed, documented factor of 2.5 kg CO2e per kg of food rescued, and it reports food that still went to waste.

Potential impact

- **Who it serves first:**
  - the **33M+ Americans with food allergies** and their families (FARE);
  - **multigenerational households** cooking for a grandparent with high blood pressure, which affects 71.6% of adults 60 and older (CDC);
  - the **~18 million food-insecure households** for whom every wasted dollar matters (USDA ERS).

  These overlap with almost every household that wastes food, which ReFED puts at nearly **$800 per person a year**.

- **Safety value.** The family's allergy and diet rules are applied to every suggestion, substitute, improvised recipe and scanned label. A description of a severe reaction gets an immediate emergency script instead of a chatty answer.
- **Money and waste.** Rescuing even a quarter of a family of four's waste is worth hundreds of dollars a year. Sous measures it in-app: dollars and kilograms rescued and CO2e avoided, plus the food that still went to waste, so the number can go down as well as up.
- **Community.** Surplus food is routed to a neighbor or community fridge before it expires (simulated listings in this build).
- **Why Alexa+:** it fits naturally upstream of Alexa+'s existing Amazon Fresh shopping. Sous decides *what* to buy, and *whether it's safe for everyone*, based on what you already have. It also fits the kitchen, where voice-first and glanceable screens matter most. The same MCP server works with any MCP client (Claude, ChatGPT, VS Code), so it isn't locked to one host.
- **Path beyond the hackathon:**
  - OAuth 2.1 + PKCE account linking (the Alexa+ checklist);
  - real grocery catalogs, and real food-bank and community-fridge directories;
  - dietitian-reviewed diet rules;
  - receipt and barcode pantry import;
  - Fire TV and Echo Hub layouts, which the MCP Apps views already scale to.

Quality of the idea: the judges' creative signals, one by one

Alexa+ creative signal (from the rules)	How Sous does it
Agentic workflow that orchestrates across services	"Plan dinners for the next three nights" chains planner → plan → grocery cart in one turn. `budget_meal_plan` and `reorder_staples` combine pantry, budget and store. With Bedrock, the model picks the chain itself.
Context-aware, state across sessions	Allergies, diets, dislikes, pantry, plan, budget, leftovers, shopping list and cook progress all persist. A brand-new session still knows Leo's allergy.
Purchasing	Token-bound cart, explicit "yes", idempotent replay, a host-side consent gate and budget awareness
Media support (cards, carousels)	Five MCP Apps views on Echo Show: recipe carousel, cook-along, cart/receipt, pantry and impact chart
MCP Apps	`ui://sous/*.html` (`text/html;profile=mcp-app`) linked from tools; cards call tools back on tap, so voice and touch share one path
Agent Skills	`skills/sous-kitchen/SKILL.md` packages the voice contract and workflows for any agent
Not a wrapper or single-turn bot	44 tools over a deterministic domain core (planner, allergen and diet guards, emergency red flag, purchase gate) with 254 automated tests

What we built during the submission window

Everything in this repository was built on **23 October 2026**, within the submission window: the MCP server and all tools, the planner and guards, the MCP Apps views, the Echo Show simulator host, the Bedrock agent, the landing page, the tests and the docs. Nothing predates the hackathon.

How we built it

- Node.js 20+, TypeScript, Express 5, `@modelcontextprotocol/sdk` 1.32 for the server and the agent's client, and Zod 4 for schemas.
- Amazon Bedrock Converse API via `@aws-sdk/client-bedrock-runtime`.
- Vanilla HTML/CSS/JS for MCP Apps views, so each view is a single self-contained file. Three.js for the landing hero.
- **254 Vitest tests**, covering:
  - planner ranking, and allergen, dislike and diet exclusion;
  - the allergy-emergency red flag, including negations, and label scanning with "may contain";
  - cart and checkout: token, expiry, idempotent replay;
  - budget, leftovers, donations, planning and household tools;
  - MCP over real Streamable HTTP;
  - the Bedrock tool loop against a scripted Converse mock;
  - the consent gate that blocks a model-initiated purchase;
  - offline small talk.
- A small **tool registry**, so each module (core, safety, budget, planning, household) registers its tools and voice routes independently. That let us build the modules in parallel.
- AI pair-programming with Claude Code for scaffolding and parallel workstreams.

Challenges

- **Testing without the real Alexa+ toolkit.** The official web simulator and inspector are behind the Private Preview, so we implemented the MCP Apps host side ourselves, following the spec (stable 2026-01-26).
- **Safety text that helps without diagnosing.** The allergy-emergency script and the food-safety answers are fixed, conservative strings in code. Sous never declares food "safe", and it always errs toward "call 911" or "throw it out".
- **Voice-first purchasing.** We had to make "yes" safe: a token-bound confirmation, plus a `followUp` contract so the agent re-issues the exact call.
- **Serverless MCP.** Streamable HTTP sessions don't survive serverless instances, so the Vercel deployment runs the stateless mode and the simulator's agent uses the SDK's in-memory transport.
- **Designing for a 768×480 base canvas** that scales 1.667× on an Echo Show 15. We had to fit huge type and low content density into it.

What's next

- Account linking (OAuth 2.1 + PKCE) and a real Amazon Fresh cart handoff.
- Real community-fridge and food-bank directories, and dietitian-reviewed diet rules.
- Receipt and photo pantry capture.
- Shared household plans and allergy profiles across Echo devices and the Alexa app, so a babysitter or grandparent's Echo knows Leo's allergy too.

Built with

`model-context-protocol` · `mcp-apps` · `streamable-http` · `alexa-plus` · `amazon-bedrock` · `aws-sdk-js-v3` · `typescript` · `node.js` · `express` · `zod` · `three.js` · `web-speech-api` · `vitest`

Tracks and mini challenges

- **Primary track: Alexa+.** Self-hosted MCP server, spec 2025-11-25, Streamable HTTP, plus a simulated Alexa+ experience on an Echo Show (the simulator source is in the repo).
- **Mini challenge: AWS Builder.** Amazon Bedrock (details below and in Product Feedback).
- **Mini challenge: Open Source** (optional, see below).

AWS Builder: services used and how

1. **Amazon Bedrock Converse API with tool use** is the Alexa+ simulation agent (`src/agent/`). The flow:

    1. The server lists its own MCP tools over Streamable HTTP and converts each `inputSchema` into a Bedrock `toolSpec` .
    2. It calls `ConverseCommand` with the household context and the server instructions as the system prompt.
    3. It executes each `toolUse` as an MCP `tools/call` and feeds back `toolResult` blocks until the model returns the final spoken answer.

    Every call is visible in the MCP trace drawer.

    A **circuit breaker** pauses Bedrock for 5 minutes after an auth, access or model error, and the deterministic engine answers instead. A misconfigured key therefore never makes a voice turn hang.

2. **Amazon Bedrock inside a tool.** `chef_improvise` asks a Bedrock model to invent a recipe from the at-risk ingredients. The output is parsed to JSON and then passes the **deterministic allergen guard** before Sous will show or speak it. If Bedrock is unconfigured or times out, a template recipe is used instead. `structuredContent.data.suggestions[0].engine` records which path ran.

3. **Auth and config.** Configure via `.env` : either a Bedrock API key (`AWS_BEARER_TOKEN_BEDROCK`) or the standard AWS credential chain, plus `AWS_REGION` and `BEDROCK_MODEL_ID` (default: the Amazon Nova Pro inference profile `us.amazon.nova-pro-v1:0`). `SOUS_BEDROCK=auto|off` sets the mode. Each Bedrock call has a hard time budget (`BEDROCK_TIMEOUT_MS` , default 8 s), because a voice turn can't hang. The agent allows at most 4 tool rounds per turn.

Open Source (optional entry)

- **Repository URL:** https://github.com/johnbosco-j/sous (public, MIT license visible in the About section).
- **GitHub username:** johnbosco-j
- **Contribution URL:** `<https://github.com/johnbosco-j/sous/tree/main/web/device>` . Replace this if you publish the simulator as its own repo.

- **What we did:** we open-sourced a reusable **Echo Show-style MCP Apps host** for Alexa+ add-on developers. It renders any `ui://` view in a sandboxed iframe and implements the host side of the MCP Apps protocol (`ui/initialize`, `tool-input`, `tool-result`, proxied `tools/call`). It adds an Alexa light bar, voice I/O and an MCP trace drawer.
- **Why it matters:** the official Alexa+ simulator is in Private Preview, so most developers have no way to see their add-on's MCP Apps cards on an Echo Show-sized screen. This host lets anyone iterate on views locally.

Product feedback (required, one block per tool)

1. Model Context Protocol spec 2025-11-25 + TypeScript SDK (`@modelcontextprotocol/sdk` 1.32)

- **Used for:** the Sous server (tools, resources, `outputSchema` / `structuredContent`, annotations, server `instructions`) and the agent's client, which connects to the server over Streamable HTTP.
- **What worked well:**
  - `StreamableHTTPServerTransport` plus Express gave us sessions (`Mcp-Session-Id`) with very little code.
  - `structuredContent` + `outputSchema` are exactly what a voice agent needs: short speech plus machine-readable data.
  - Tool annotations (`openWorldHint` + `idempotentHint` on checkout, `readOnlyHint` on queries) map naturally to "ask before buying".
- **What needs work:**
  - The spec has moved past 2025-11-25: the 2026-07-28 revision removes protocol sessions. The hackathon says "2025-11-25 or later, once confirmed", and it is unclear which later revisions Alexa+ accepts.
  - Tool input schemas generated from Zod are valid JSON Schema, but they are not directly portable to LLM tool APIs (see Bedrock below). A "portable schema" profile in the spec would help every agent host.
- **Onboarding:** smooth. We used the low-level `Server` API so one tool table emits `inputSchema`, `outputSchema`, annotations, icons and `_meta.ui` uniformly. The SDK's shared types (`Tool`, `CallToolResult`) let our server and our in-process Alexa+ stand-in client share one contract.
- **Would we build with it again?** Yes. It is the cleanest way to make one backend work with Alexa+ and every other agent host.

2. MCP Apps extension (`io.modelcontextprotocol/ui`, spec 2026-01-26)

- **Used for:** five Echo Show views (pantry grid, meal carousel, cook mode, cart/checkout, impact chart) linked from tools through `_meta.ui.resourceUri`, served as `text/html;profile=mcp-app`.
- **What worked well:**
  - A single self-contained HTML file per view.
  - Tool results arrive as standard `CallToolResult`s, so the view code is the same data the model sees.
  - `tools/call` from the view lets touch and voice share one code path.
- **What needs work:**
  - Easy-to-miss host rules: `tool-input` must precede `tool-result`, and nothing may be sent before `initialized`.

    - The restrictive default CSP blocks web fonts unless you declare `_meta.ui.csp.resourceDomains`.
    - The deprecated flat `_meta["ui/resourceUri"]` still appears in samples.
- **Onboarding:** the spec is thorough, but there is no tiny "hello view" with no SDK.
- **Would we build with it again?** Yes. Cards and carousels on Echo Show are the difference between a voice bot and a product.

3. Alexa+ developer resources (MCP Toolkit docs, MCP add-on design guide, visual foundations)

- **Used for:** design constraints (768×480 base canvas, 1.667× scale on Echo Show 15, low density, inline vs. fullscreen modes), the transport requirement (Streamable HTTP, spec 2025-11-25), and the account-linking checklist we documented as our production path.
- **What worked well:**
    - The design guide is concrete: the canvas size, a 1.6× type scale, and "you can't script what Alexa says, design your data".
    - The checklist for OAuth (401 without `WWW-Authenticate`, RFC 9728 metadata, S256 PKCE) is precise.
- **What needs work:**
    - The MCP Toolkit web simulator and local inspector require Private Preview access, so hackathon builders cannot see their add-on on a real Alexa+ surface.
    - We could not find a statement of which MCP Apps host capabilities Alexa+ supports (view-initiated `tools/call`, `ui/open-link`, display-mode requests) or whether MCP prompts are surfaced.
- **Onboarding:** the docs were easy to read, but the "hello world on a real device" step was not available to us.
- **Would we build with it again?** Yes, especially once the toolkit is open to everyone.

4. Amazon Bedrock: Converse API with tool use (`@aws-sdk/client-bedrock-runtime`, AWS SDK for JavaScript v3)

- **Used for:** the Alexa+ simulation agent loop (tool use over MCP tools) and the `chef_improvise` recipe generator.
- **What worked well:**
    - The Converse API's model-agnostic `toolSpec` / `toolUse` / `toolResult` shape maps almost 1:1 onto MCP `tools/list` and `tools/call`. We switch models by changing `BEDROCK_MODEL_ID`.
    - Bedrock API keys (`AWS_BEARER_TOKEN_BEDROCK`) are much simpler for hackathon setups than IAM users.
- **What needs work:**
    - **Schema portability.** Different Bedrock models accept different subsets of JSON Schema in `toolSpec.inputSchema.json`, and we found no single documented portable subset. We strip MCP/Zod schemas down to `type`, `properties`, `required`, `description`, `enum` and `items` before sending them.
    - **Credential errors.** Our cloud dev container had AWS keys in its environment that Bedrock rejected: `UnrecognizedClientException: The security token included in the request is invalid`. The error doesn't say which credential source the SDK used (env keys, profile, or a Bedrock API key). Every voice turn paid a failed round-trip until we added a circuit breaker.
    - **Docs.** We found Python and cURL examples for API-key auth, but no official JavaScript v3 example combining a bearer token with Converse tool use.

    - **Credits.** The hackathon's AWS credit codes ran out on Oct 7, so late joiners had to use their own account or the free tier.
- **Onboarding:** good. Converse is the right abstraction.
- **Would we build with it again?** Yes. Bedrock with MCP is a natural pair for Alexa+ add-ons.

5. Web Speech API (simulator voice I/O)

- **Used for:** push-to-talk recognition and spoken replies in the Echo Show simulator.
- **What worked well:** zero-dependency TTS and ASR in Chrome.
- **What needs work:** not available in every Chromium browser (Brave does not expose recognition), so we added a text box and suggestion chips.
- **Would we build with it again?** Yes, for prototyping only. Real Alexa+ replaces it.

Feature requests

#	Request	Why it matters for Sous	Priority
1	Open the Alexa+ MCP Toolkit web simulator and local inspector to all hackathon and registered developers, at least in a read-only "render my MCP Apps views" mode	We had to build our own Echo Show host to see our cards; every team is duplicating this	**Critical**
2	Publish an "MCP Apps on Alexa+" compatibility table: supported host capabilities, display modes, view-initiated `tools/call`, CSP allowances, max view size, prompts support	We could not confirm whether buttons in our cards can call tools on real devices	**Critical**
3	A purchasing hand-off for add-ons: let an MCP add-on propose a cart that Alexa+ checks out through the customer's Amazon Fresh/Whole Foods account with Alexa's own confirmation UX	Sous decides *what* to buy, and Alexa+ should own *how* to pay. Today we can only simulate checkout	**Important**
4	Official JS/TS sample: Bedrock Converse tool-use loop over an MCP client (with API-key auth)	It is the glue every Alexa+ and AWS Builder team writes by hand	**Important**
5	Kitchen-timer and list primitives exposed to add-ons (start an Alexa timer, add to an Alexa shopping list)	Our cook mode re-implements timers that Alexa already has natively	**Important**
6	Clarify which MCP spec versions after 2025-11-25 Alexa+ accepts (e.g. 2026-07-28 sessionless)	Avoids building against a transport behavior Alexa+ may not support	**Nice-to-have**
7	Bedrock Converse: publish the portable JSON Schema subset for `toolSpec` (or normalize schemas server-side)	Every MCP → Bedrock bridge must currently guess which keywords each model accepts	**Important**

#	Request	Why it matters for Sous	Priority
8	Bedrock auth errors that name the credential source used (env keys / profile / API key)	We lost time to an `UnrecognizedClientException` from invalid credentials in the environment, with no hint which source the SDK had picked	**Nice-to-have**

Testing instructions (for judges)

Try it live, no install: https://sous-alexa.vercel.app/device (Echo Show simulator; type or tap the suggestion chips, press T for the MCP trace). Landing page: https://sous-alexa.vercel.app. Public MCP endpoint (stateless Streamable HTTP, spec 2025-11-25): `https://sous-alexa.vercel.app/mcp`. Try it with `npx @modelcontextprotocol/inspector`.

```
git clone https://github.com/johnbosco-j/sous && cd sous
npm install
npm start                       # http://localhost:8787
```

- **Echo Show simulator:** http://localhost:8787/device. Click the suggestion chips in order, or type or speak (Chrome). Press `t` for the MCP trace drawer.
- **Landing page:** http://localhost:8787/
- **Raw MCP endpoint:** http://localhost:8787/mcp (Streamable HTTP). Try `npx @modelcontextprotocol/inspector`, choose transport "Streamable HTTP", URL `http://localhost:8787/mcp`.
- **Optional Bedrock:** copy `.env.example` to `.env`, then set `AWS_REGION`, `BEDROCK_MODEL_ID`, and either `AWS_BEARER_TOKEN_BEDROCK` or AWS credentials. Without them, the deterministic router and template recipes are used, and the badge in the trace drawer shows `rules`.
- **Health check:** `GET /healthz` reports the protocol version, transport, tool count and Bedrock status.
- **Reset demo data:** the "Reset demo" control in the simulator, or `POST /api/reset`.
- **Tests:** `npm test`
- All demo data is synthetic. Orders are simulated, and nothing is purchased.

Links

- Repo: https://github.com/johnbosco-j/sous (MIT)
- Live demo: https://sous-alexa.vercel.app/device
- Demo video: `<YouTube URL>`

Friction log

Every entry below happened while building Sous on 23 Oct 2026, or can be checked against the linked public docs. Severity scale: **Blocker / High / Medium / Low**.

F1: Testing a self-hosted MCP server on a real Alexa+ surface

- **Task:** See Sous's tools and MCP Apps cards running inside Alexa+ on an Echo Show before submitting.
- **Steps:**
  1. Read the Alexa+ MCP QuickStart and the "Alexa+ add-on development stages" page.
  2. Found that the flow starts at *Discover* with a **Private Preview request**, after which the `alexa-ai` CLI, the web simulator and the local inspector become available.
- **Expected:** Hackathon participants in the Alexa+ track can connect a self-hosted MCP server to some Alexa+ test surface: a simulator, an inspector or a device.
- **Actual:** No public Alexa+ simulator was available to us. The official toolkit is gated behind the preview, and the rules allow a "simulated Alexa+ experience" instead, so each team builds its own stand-in.
- **Severity:** High. We could not verify behavior on the real host.
- **Workaround:** Built a local Echo Show-style MCP Apps host (`/device`). Its Amazon Bedrock agent loop calls our server as an MCP client over Streamable HTTP, the way Alexa+ would.
- **Suggestion:** Give registered hackathon participants time-boxed preview access, or publish the local inspector (render-only, no certification) on npm for everyone.

F2: Finding the Alexa+ MCP docs from the hackathon Resources page

- **Task:** Find the Alexa+-specific requirements for MCP add-ons: transport, auth, visual design.
- **Steps:** Opened the hackathon Resources page. The Alexa+ section links to "Build with Agent Skills" (modelcontextprotocol extensions site) and the MCP Streamable HTTP spec page. We then searched the web and found the Alexa+ MCP Toolkit QuickStart, the MCP add-on Design Guide and Visual Foundations on developer.amazon.com.
- **Expected:** The Alexa+ track resources link the Alexa+ MCP Toolkit and Design Guide directly.
- **Actual:** Only generic MCP spec links are listed for Alexa+. The Alexa+-specific pages had to be found through search. They contain the 768×480 base canvas, the 1.667× Echo Show scale, the OAuth checklist and the statement that MCP Apps are supported.
- **Severity:** Medium. These pages changed our design, and we found them late.
- **Workaround:** Web search.
- **Suggestion:** Add the MCP Toolkit QuickStart, Design Guide, Visual Foundations and Account Linking checklist to the Alexa+ section of the Resources page.

F3: Knowing which MCP Apps features Alexa+ actually honors

- **Task:** Decide whether buttons inside our Echo Show cards ("Cook this", "Add to cart", "Next step") can call tools on a real Alexa+ device.
- **Steps:** Read the MCP Apps spec (stable 2026-01-26), the Alexa+ design guide and the glossary (inline, fullscreen and voice-only modes), and searched for an Alexa+ MCP Apps compatibility statement.

- **Expected:** A table of supported host capabilities: view-initiated `tools/call`, `ui/open-link`, `ui/request-display-mode`, CSP allowances and container size.
- **Actual:** We could not find one. The docs confirm MCP Apps are supported for visuals, but not which view → host requests are allowed.
- **Severity:** Medium.
- **Workaround:** Every button action is also reachable by voice, so a card still works if the host ignores `tools/call` from views.
- **Suggestion:** Publish an "MCP Apps on Alexa+" capability matrix per device class (Echo Show 5/8/15, Echo Hub, Fire TV).

F4: The MCP Apps host details live only in the ext-apps spec

- **Task:** Implement the host side of MCP Apps in our simulator and make our views render on a spec-compliant host.
- **Steps:**
  1. Wrote an internal contract: `ui/initialize` → `initialized` → `tool-result`, with Google Fonts in the views.
  2. Cross-checked it against the ext-apps spec text.
- **Expected:** The Alexa+ docs (or a short MCP Apps quickstart) list the host rules a view depends on.
- **Actual:** Two rules we had missed exist only in the ext-apps spec:
  - The host MUST send `ui/notifications/tool-input` (once, with the full arguments) **before** `tool-result`, and must send nothing before `initialized`.
  - A resource without `_meta.ui.csp` gets `default-src 'none'`, so web fonts are blocked unless `resourceDomains` declares them.
- **Severity:** Low. It was caught before shipping.
- **Workaround:** Added `tool-input` to the host. Declared the font origins in `_meta.ui.csp` and kept system-font fallbacks. Made views tolerant of message order.
- **Suggestion:** Add a "host rules your view relies on" box to the Alexa+ Visual Foundations page, and a lifecycle conformance check to the inspector.

F5: Which MCP features and spec versions Alexa+ uses

- **Task:** Decide whether to invest in MCP prompts, and which protocol revision to target.
- **Steps:** Read the Alexa+ MCP client lifecycle page, whose sample `initialize` exchange advertises only `tools` and `resources`. The hackathon rules say 2025-11-25 "or a later version, once confirmed", and the MCP changelog shows a newer 2026-07-28 revision that removes protocol-level sessions.
- **Expected:** A "supported MCP features and versions" statement.
- **Actual:** Not stated, so we inferred that prompts are likely ignored. It is unclear whether a sessionless (2026-07-28) server would work.
- **Severity:** Low.
- **Workaround:** All behavior lives in tools and server `instructions` (prompts are a bonus). We targeted exactly 2025-11-25 with Streamable HTTP sessions.
- **Suggestion:** Add a features-and-versions checklist to the Supported Capabilities page.

F6: Invalid AWS credentials made every Bedrock voice turn pay a failed round-trip

- **Task:** Run the Alexa+ stand-in agent on Amazon Bedrock (Converse with tool use) from our cloud dev container.

- **Steps:** Started the server with `SOUS_BEDROCK=auto`. The container already had AWS keys in its environment, so the SDK picked them up.
- **Expected:** Either the call works, or it fails once with a clear message about which credential source was used.
- **Actual:** Every turn failed with `UnrecognizedClientException: The security token included in the request is invalid`. The error doesn't say whether it came from environment keys, a profile, or a Bedrock API key. Each voice turn waited on the failed call before falling back.
- **Severity:** Medium.
- **Workaround:** Added a circuit breaker. After an auth, access or model error, Bedrock is skipped for 5 minutes and the deterministic engine answers immediately.
- **Suggestion:** Include the resolved credential source in SDK auth errors, and document the precedence between `AWS_BEARER_TOKEN_BEDROCK` and SigV4 credentials on the API-keys page.

F7: JSON Schema portability for Converse `toolSpec`

- **Task:** Expose every MCP tool (`tools/list`) to Bedrock as a Converse `toolSpec`.
- **Steps:** Passed the MCP `inputSchema` (JSON Schema generated by Zod 4) as `toolSpec.inputSchema.json`.
- **Expected:** One documented, portable JSON Schema subset that every Converse tool-use model accepts.
- **Actual:** Bedrock models accept different subsets of JSON Schema, and we found no single portable list in the docs.
- **Severity:** Medium.
- **Workaround:** Strip each schema to `type`, `properties`, `required`, `description`, `enum` and `items` before sending.
- **Suggestion:** Document the supported keyword subset per model family in the Converse tool-use guide, or have Converse normalize schemas server-side. An "MCP tools → Converse" helper in the SDK would remove this step for every Alexa+ and AWS Builder team.

F8: AWS credits for the AWS Builder mini challenge

- **Task:** Get AWS promotional credits for Amazon Bedrock usage.
- **Steps:** Read the Rules and Resources pages.
- **Expected:** Credits available until the Oct 21 request deadline stated in the rules.
- **Actual:** "October 7th Update: … we are out of credit codes and will no longer be accepting credit requests."
- **Severity:** Medium for late joiners.
- **Workaround:** Bedrock is optional in Sous. A deterministic router and template recipes keep the demo fully functional without AWS.
- **Suggestion:** Offer capped AWS Workshop Studio or sandbox accounts for hackathon builders instead of one-time credit codes.

F9: Bedrock API keys + Converse tool use from JavaScript

- **Task:** Authenticate the Converse tool-use loop with a Bedrock API key from Node.js.
- **Steps:** Searched the AWS docs for API-key usage. They show the `AWS_BEARER_TOKEN_BEDROCK` environment variable with Python (boto3) and cURL examples.
- **Expected:** An official AWS SDK for JavaScript v3 example: `ConverseCommand` with `toolConfig`, authenticated by the API key.

- **Actual:** We found no official JS example combining the two.
- **Severity:** Low.
- **Workaround:** Relied on the SDK reading the environment variable, with the standard credential chain as a fallback.
- **Suggestion:** Add a JS/TS tab to the "Use an Amazon Bedrock API key" page and a Converse tool-use sample that loops over an MCP client.

F10: Getting the code to GitHub from a cloud AI dev environment (non-Amazon tooling)

- **Task:** Create and push the public repository required for submission.
- **Steps:**
  1. The cloud coding session's GitHub token was invalid, so the repo could not be created from the session.
  2. Created `johnbosco-j/sous` manually on GitHub.
  3. Pushing returned HTTP 403 until the Claude GitHub App was installed on the new repository.
- **Expected:** Push works once the repo exists.
- **Actual:** An extra app installation was needed for the new repo.
- **Severity:** Medium on deadline day.
- **Workaround:** Installed the GitHub App on the repository.
- **Suggestion (for the hackathon checklist):** Remind builders who use AI coding agents to grant repo access *before* deadline day, alongside the existing "add collaborators" reminder.

F11: Hosting a Streamable HTTP MCP server on serverless (Alexa+ needs a public HTTPS endpoint)

- **Task:** Give judges, and Alexa+, a public HTTPS MCP endpoint by deploying Sous to a serverless host (Vercel).
- **Steps:**
  1. Deployed the same Express app that runs locally with stateful Streamable HTTP. That mode keeps an in-memory session map keyed by `Mcp-Session-Id`.
  2. Read the SDK transport options.
  3. Switched to the stateless mode (`sessionIdGenerator: undefined`, a fresh server per POST, JSON responses).
  4. Moved the simulator's agent from a self-HTTP connection to the SDK's in-memory transport.
- **Expected:** One recommended hosting pattern for Alexa+ add-ons, telling you whether Alexa+ keeps `Mcp-Session-Id` sessions and whether stateless servers are fine.
- **Actual:** Stateful sessions can't be relied on when requests can land on different serverless instances. The Alexa+ docs we could reach say Streamable HTTP is required, but not which session behavior Alexa+ expects from the server.
- **Severity:** Medium.
- **Workaround:**
  - Stateless Streamable HTTP on Vercel (`/healthz` reports `mode: stateless`), with sessions kept for local runs.
  - Demo state persists per instance in `/tmp`.
- **Suggestion:** Add a "hosting your MCP add-on" page covering:
  - stateful vs. stateless Streamable HTTP;
  - whether Alexa+ resumes sessions;

- a reference deployment on AWS Lambda or App Runner with a durable session store such as DynamoDB.